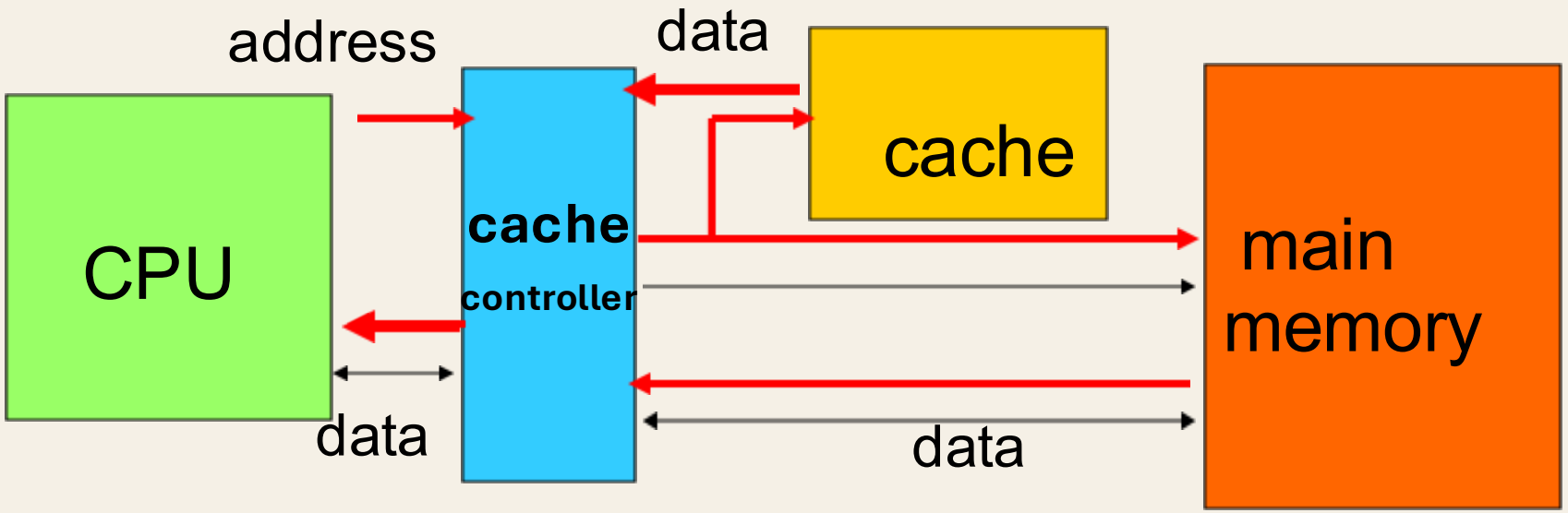
Embedded System HW Design
Week4
Caches and Memory Management Units (MMU)

- **Identify different forms of Cache algorithms**
- **Discuss the advantages of having a MMU**
- Types of embedded memory

Template designed by Tim Palmer tpalmer@vsu.edu

Embedded System HW Design
Week4
Cache Sub System



Embedded System HW Design
Week4
Cache Operation

- Caches used to speed up memory system performance
- caches are used for:
  - instructions
  - data
  - data + instructions
- Not all memory locations are cached, hence memory access time is no longer deterministic

Embedded System HW Design
Week4
Cache Terms Used

- Cache Hit: Required location in cache
- Cache Miss: Required location not in cache
- Working Set: Set of locations used by CPU

Embedded System HW Design
Week4
Types of Cache Misses

- Compulsory Miss (Cold Miss)
  Location has never been accessed

- Capacity Miss
  Working set is too large

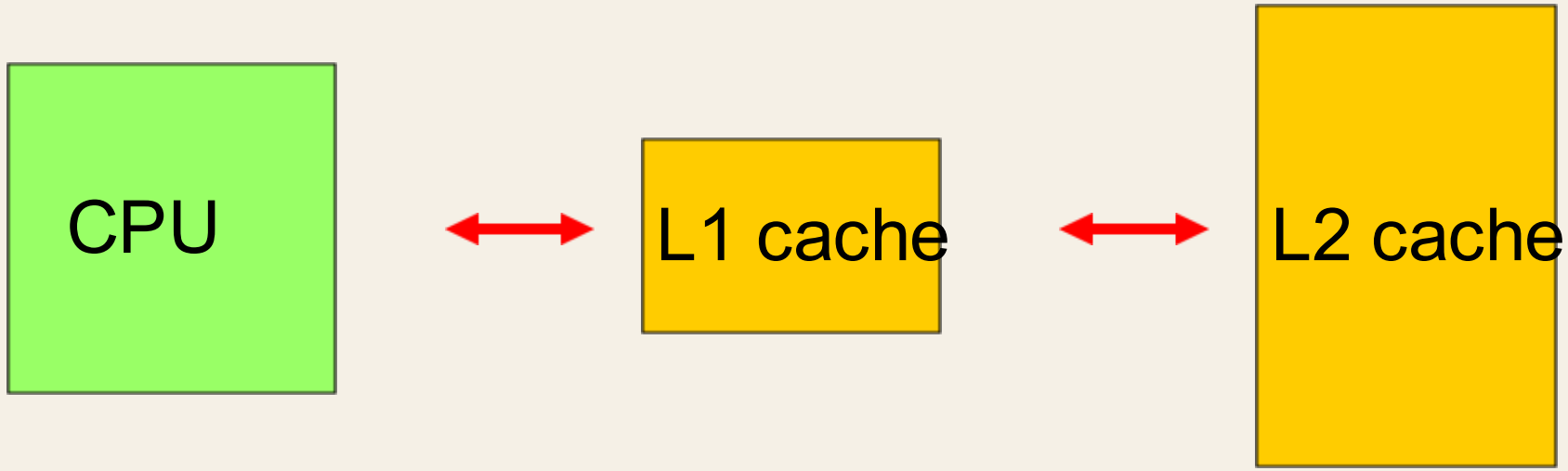
- Conflict Miss
  Multiple locations in **Working Set** map to same cache entry

Embedded System HW Design
Week4
Caches Memory System Performance

- h = cache hit rate
- t_{cache} = cache access time

  t_{main} = main memory access time
- Average memory access time:
  - $t_{av} = ht_{cache} + (1-h)t_{main}$

Embedded System HW Design
Week4
Multiple level of Cache



Embedded System HW Design
Week4
Multi-level Cache Access Time

- h_1 = Cache Hit rate on L1
- h_2 = Cache Hit rate on L2
- Average memory access time:
  - $t_{av} = h_1 t_{L1} + (h_2 - h_1) t_{L2} + (1 - h_2 - h_1) t_{main}$

Embedded System HW Design
Week4
Cache Performance Benefits

- Keep frequently-accessed locations in fast cache
- Cache retrieves more than one word at a time
  - Sequential accesses are faster after first access

Embedded System HW Design
Week4
Caches Replacement Policies

- Replacement Policy: Strategy for choosing which cache entry to throw out to make room for a new memory location

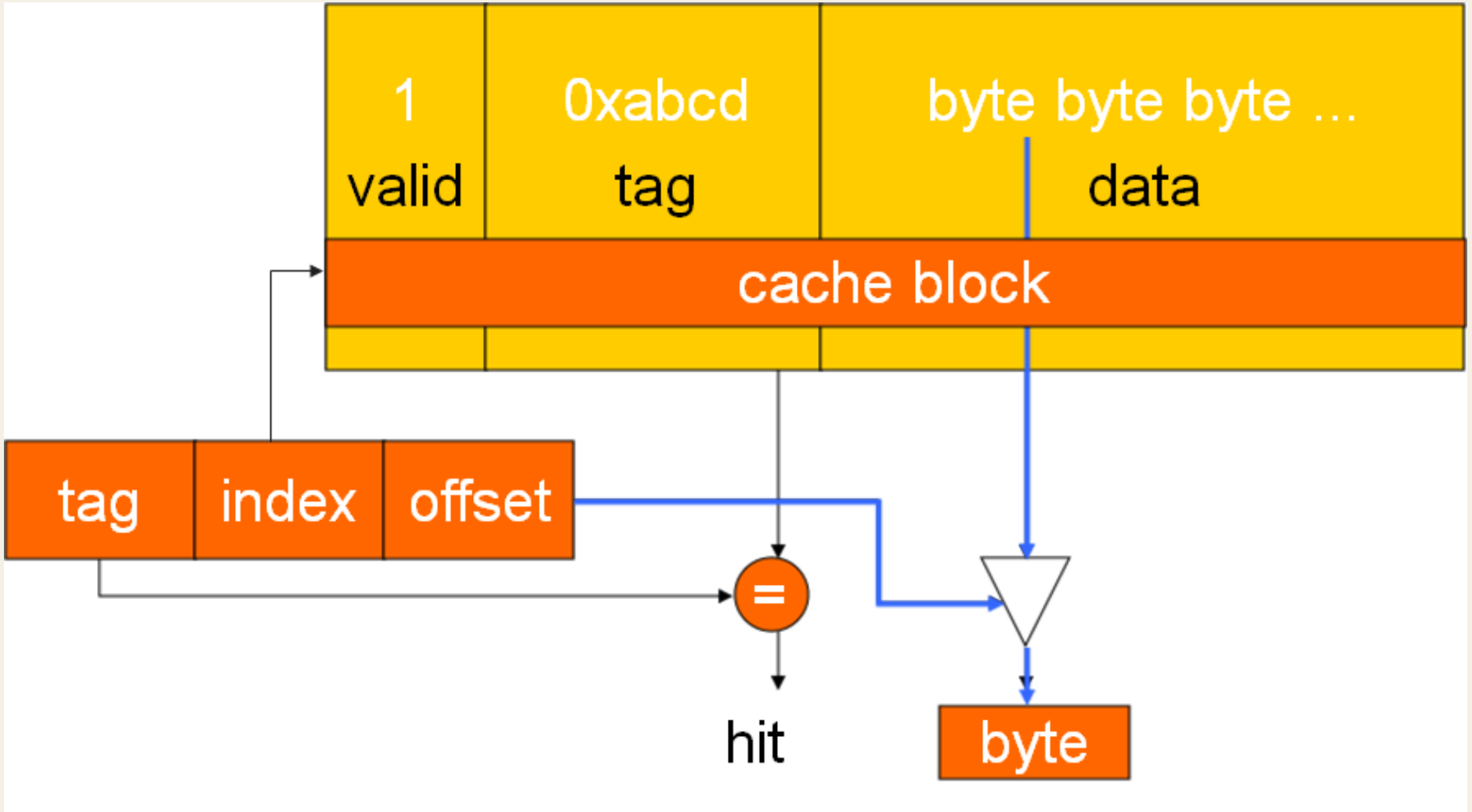
- Two popular strategies:
  - Random
  - Least Recently Used (LRU)

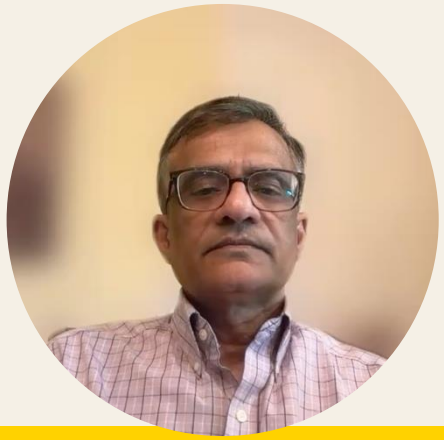
Embedded System HW Design
Week4

Cache Organizations

- Fully-associative: Any memory location can be stored anywhere in the cache (almost never implemented)
- Direct-mapped: Each memory location maps onto exactly one cache entry
- N-way set-associative: Each memory location can into one of n sets

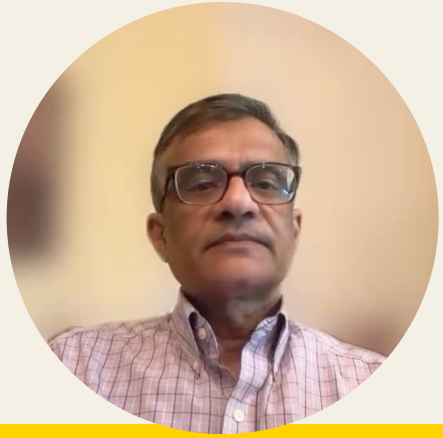
Embedded System HW Design
Week4
Direct-mapped Cache





Embedded System HW Design
Week4
Cache Write Operations

- Write-through: Immediately copy write to main memory
- Write-back: Write to main memory only when location is removed from cache

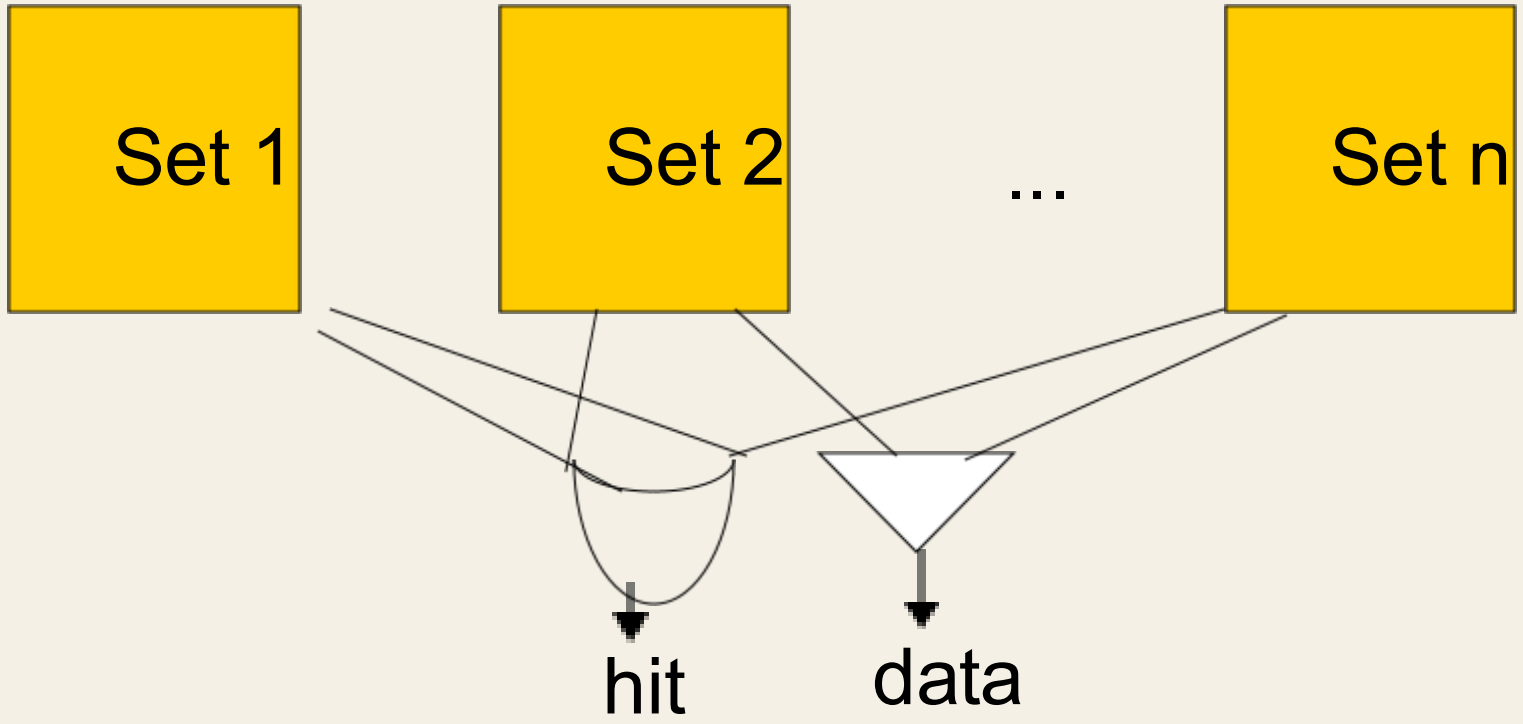


Embedded System HW Design
Week4
Direct-mapped cache locations

- Many locations map onto the same cache block
- Conflict misses are easy to generate:
  - Array a[] uses locations 0, 1, 2, …
  - Array b[] uses locations 1024, 1025, 1026, …
  - Operation a[i] + b[i] generates conflict misses.

Embedded System HW Design
Week4
Set-associative cache

A set of direct-mapped caches



Embedded System HW Design
Week4
Direct-mapped vs. set-associative

address	data
000	0101
001	1111
010	0000
011	0110
100	1000
101	0001
110	1010
111	0100

Embedded System HW Design
Week4
Direct-mapped cache behavior

tag block

- After 0 01 access:

block	tag	data
00	-	-
01.	0	1111
10	-	-
11	-	-

- After 010 access:

block	tag	data
00	-	-
01	0	1111
10	0	0000
11	-	-

Embedded System HW Design
Week4
Direct-mapped cache behavior(cont.)

- After 011 access:

block	tag	data
00	-	-
01.	0	1111
10	0	0000
11	0	0110

- After 100 access:

block	tag	data
00	1	1000
01	0	1111
10	0	0000
11	0	0110

Embedded System HW Design
Week4
Direct-mapped cache behavior(cont.)

- After 101 access:

block	tag	data
00	1	1000
01	1	0001
10	0	0000
11	0	0110

Old data

Replaced

- After 111 access:

block	tag	data
00	1	1000
01	1	0001
10	0	0000
11	1	0100

New data

Embedded System HW Design
Week4
2-way set-associative cache behavior

- Final state of cache (twice as big as direct-mapped):

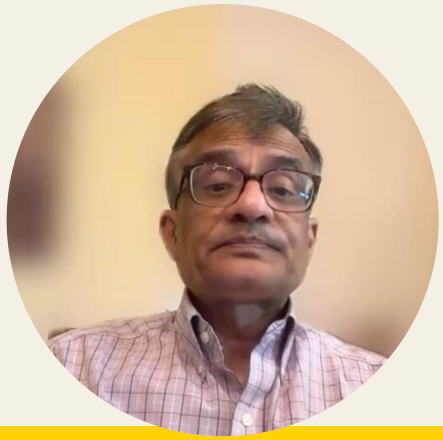
block	bank0tag	bank0data	blank1tag	bank1data
00	1	1000	-	-
01	0	1111	1	0001
10	0	0000	-	-
11	0	0110	1	0100

Address 111 data is stored in 2nd set

Embedded System HW Design
Week4
2-way set-associative cache behavior

- Final state of cache (same size as direct-mapped):

block	bank0tag	bank0data	bank1tag	bank1data
0	01	0000	10	1000
1	10	0111	11	0100

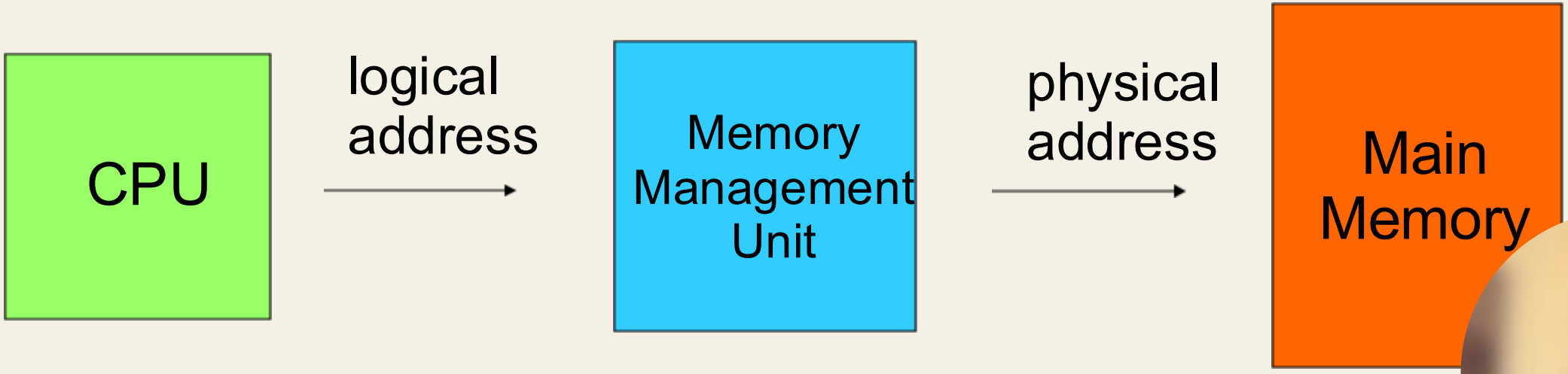


Embedded System HW Design
Week4
ARM cache Example

- StrongARM:
  - 16 Kbyte, 32-way, 32-byte block **instruction cache**
  - 16 Kbyte, 32-way, 32-byte block **data cache** (write-back)

Embedded System HW Design
Week4
Memory management units

- Memory management unit (MMU) translates addresses:



Embedded System HW Design
Week4
Memory management tasks

- Allows programs to move in physical memory during execution
- Allows virtual memory:
  - memory images kept in secondary storage;
  - images returned to main memory on demand during execution
- Page fault: request for location not reside memory

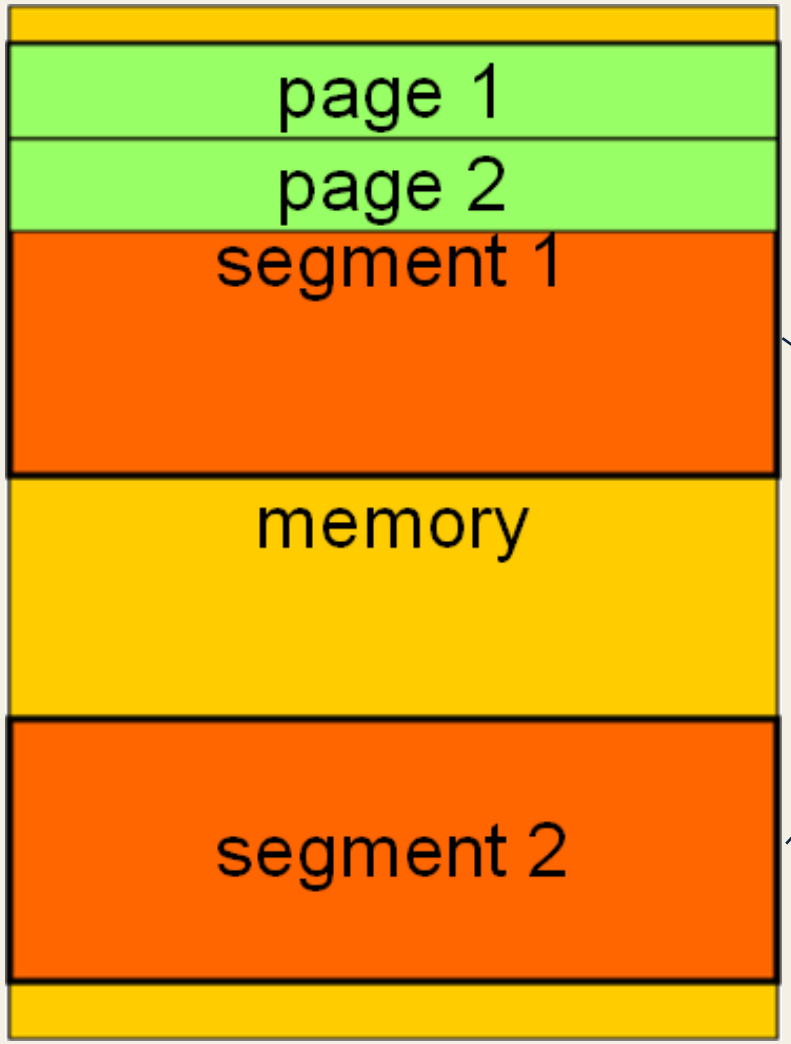
Embedded System HW Design
Week4
Address Translation

- Requires some sort of register/table to allow arbitrary mappings of logical to physical addresses

- Two basic schemes:
  - Segmented
  - Paged

- Segmentation and paging can be combi
(x86)

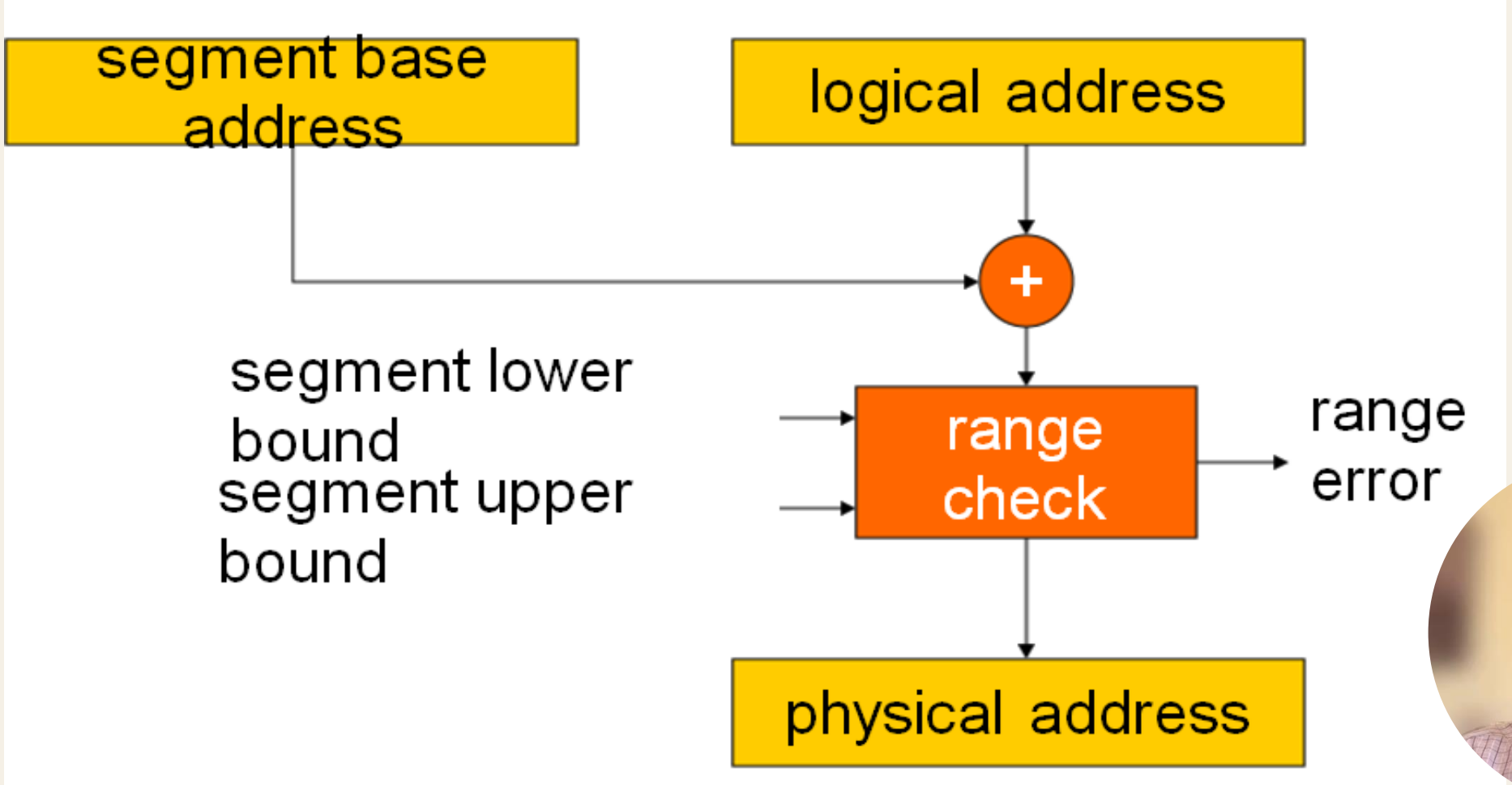
Embedded System HW Design
Week4
Segments and Pages



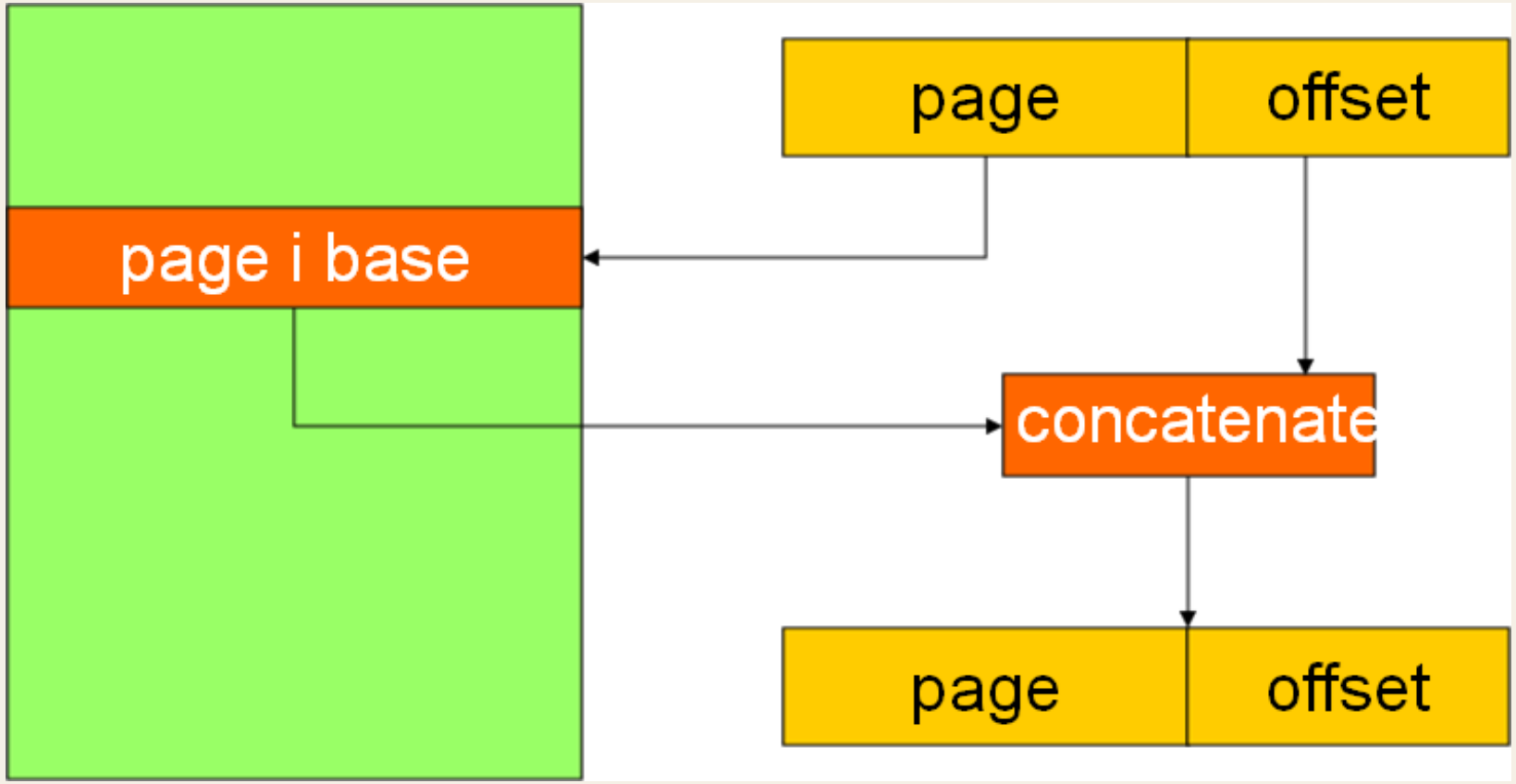
Same size

Different sizes

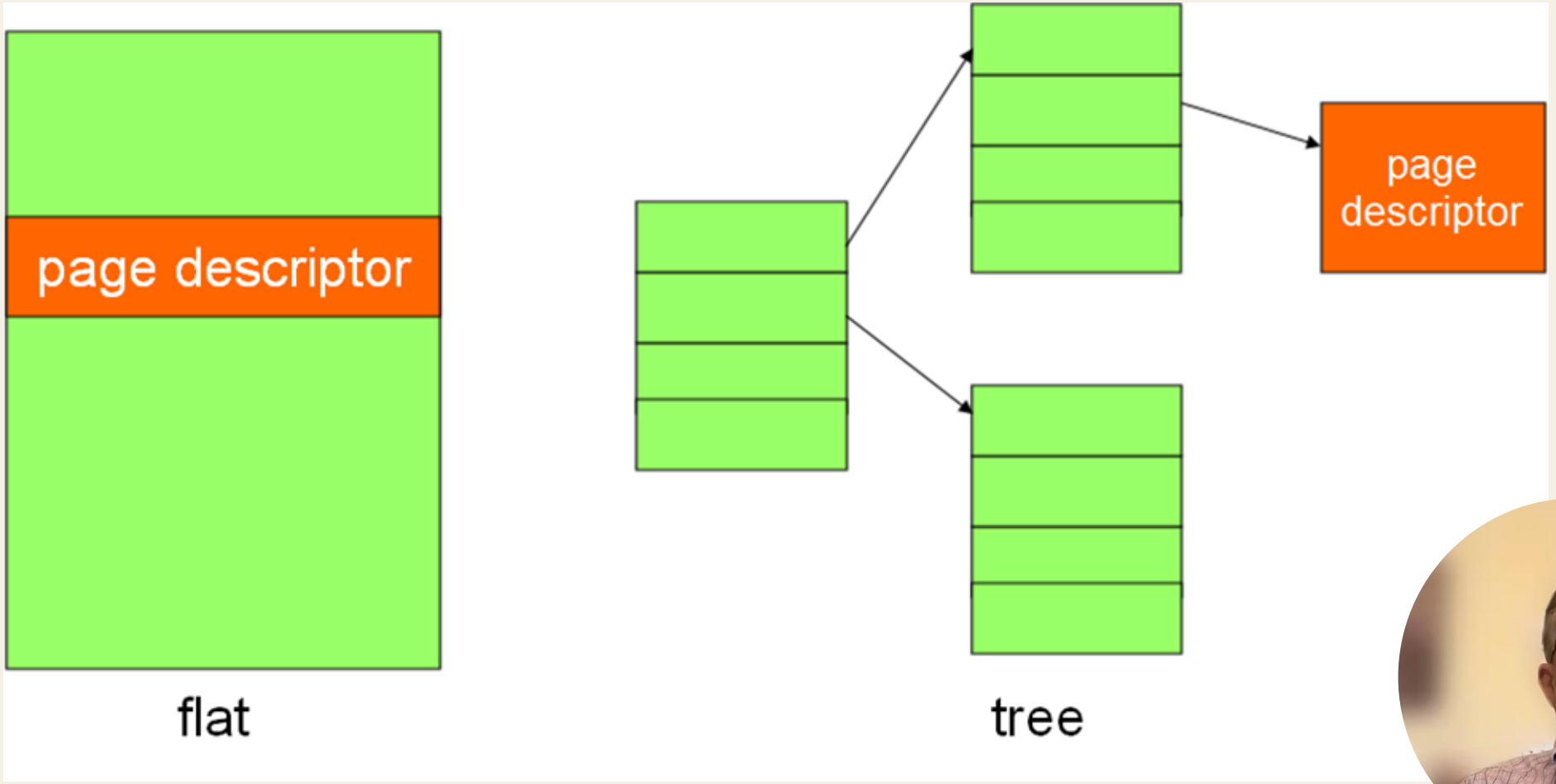
Embedded System HW Design
Week4
Segments address translation



Embedded System HW Design
Week4
Page address translation



Embedded System HW Design
Week4
Page table organization



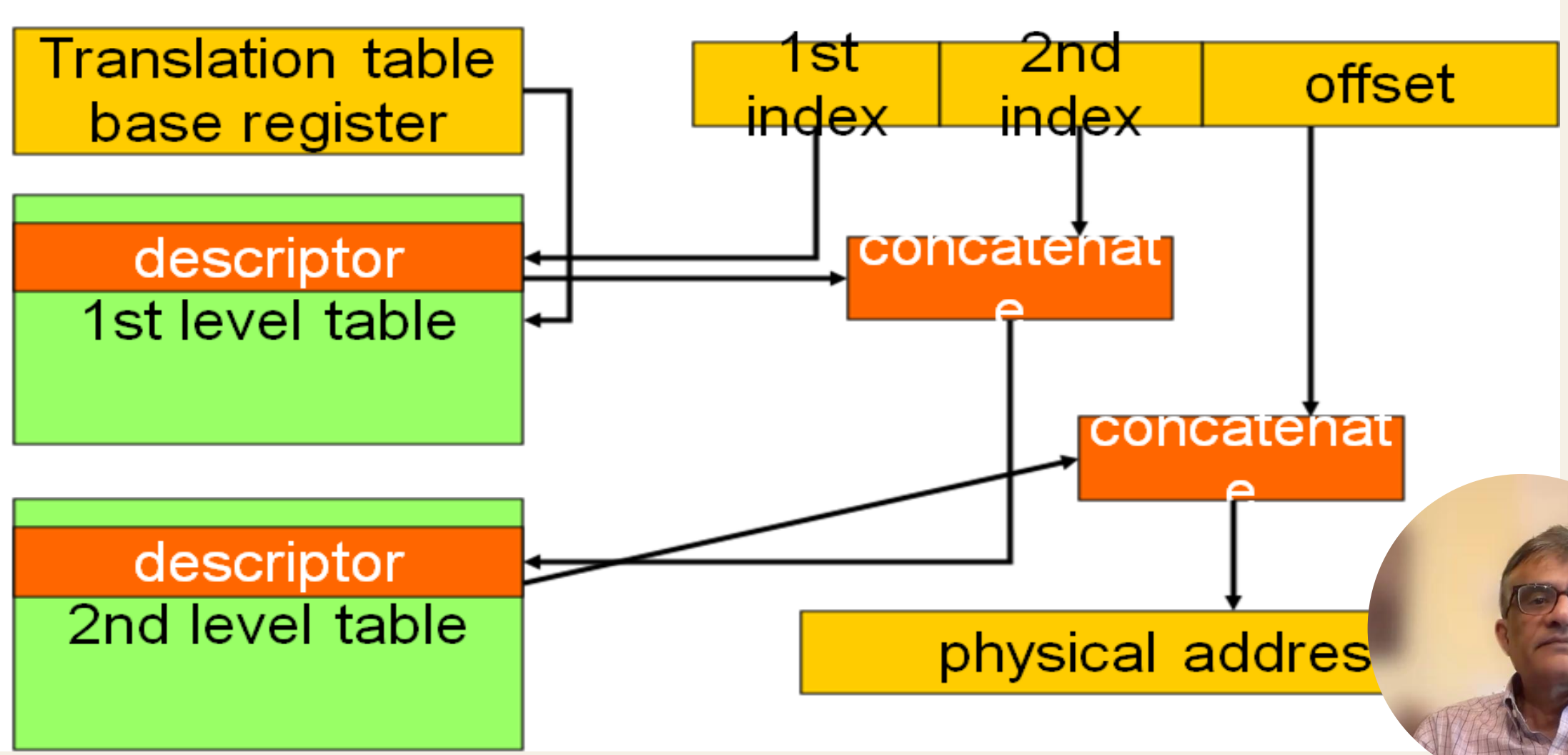
Embedded System HW Design
Week4
Caching address translation

- Large translation tables require main memory access
- TLB: cache for address translation
  - Typically small

Embedded System HW Design
Week4
ARM memory management

- Memory region types:
  - section: 1 Mbyte block
  - large page: 64 kbytes
  - small page: 4 kbytes
- An address is marked as section-mapped or page-mapped
- Two-level translation scheme

Embedded System HW Design
Week4
ARM address translation



Embedded System HW Design
Week4
Caches and Memory Management Units (MMU)

- **Identify different forms of Cache algorithms**
- **Discuss the advantages of having a MMU**
- Types of embedded memory

Template designed by Tim Palmer tpalmer@vsu.edu